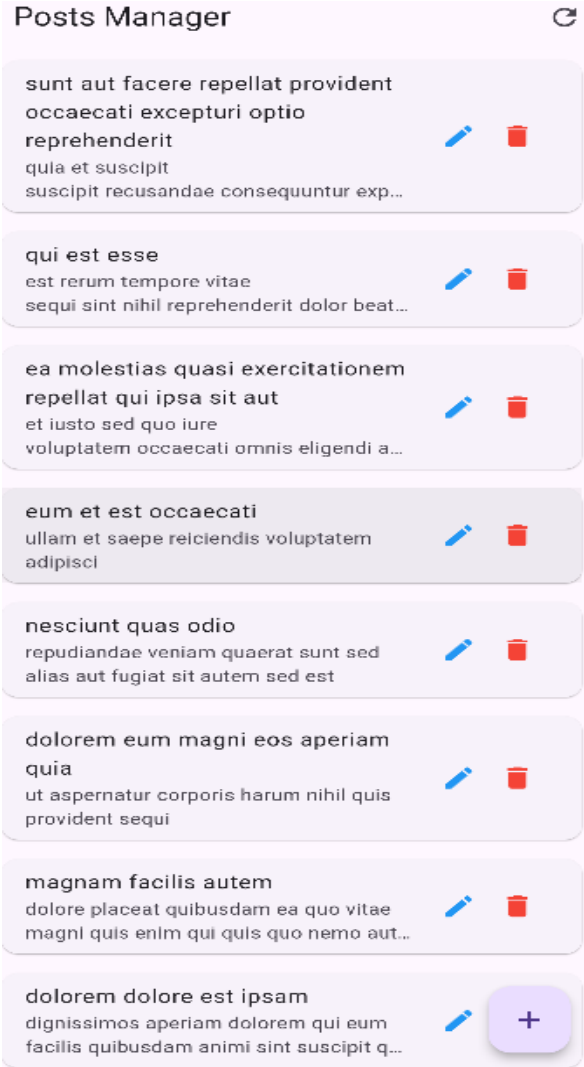
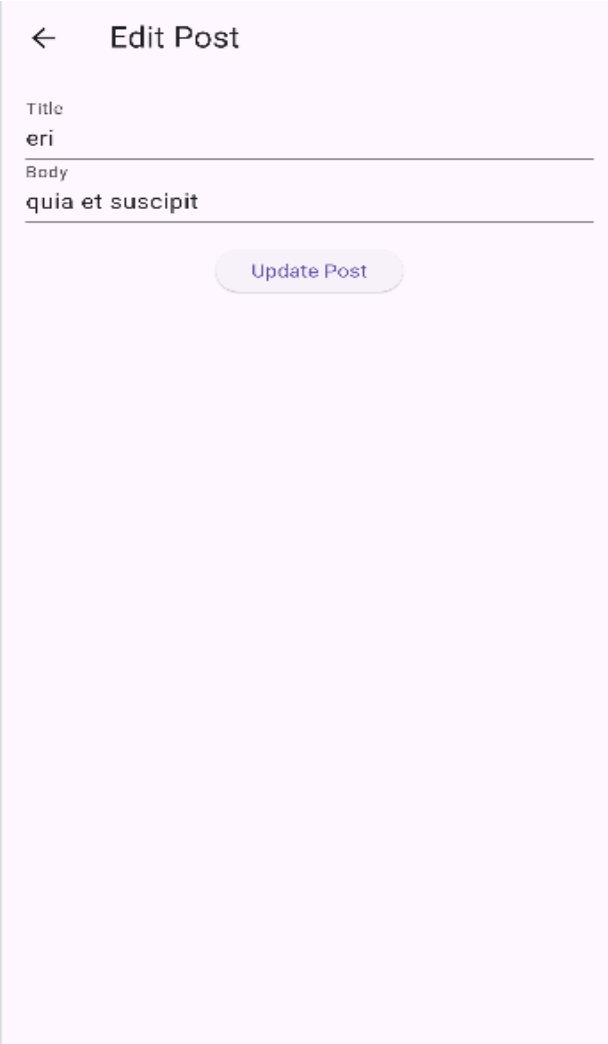
REPORTS

1. Home secreen



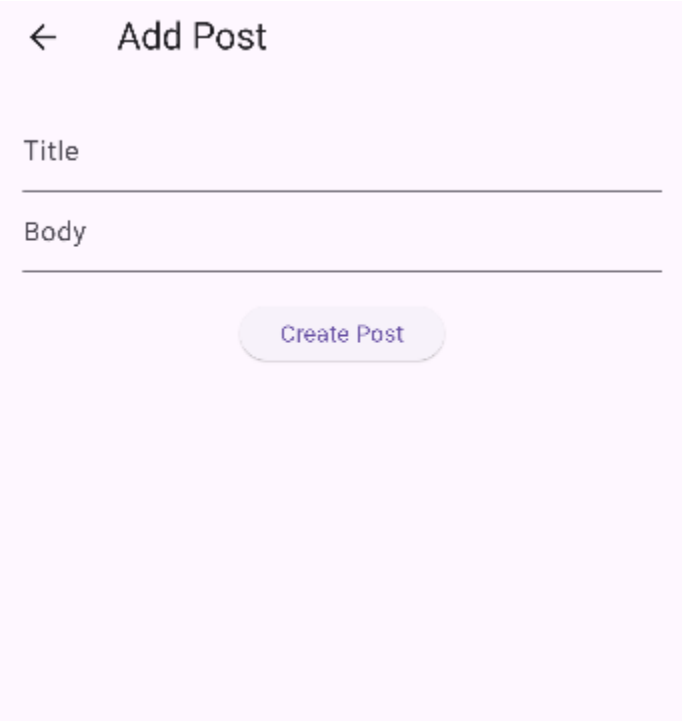
This is the Home Screen of the Lecture Manager app. It displays all lectures fetched from the API, showing the title, topic, body, and date. Users can also search lectures by title or topic. This part of the UI depends on the Future that fetches lecture data from the server

1. Edit



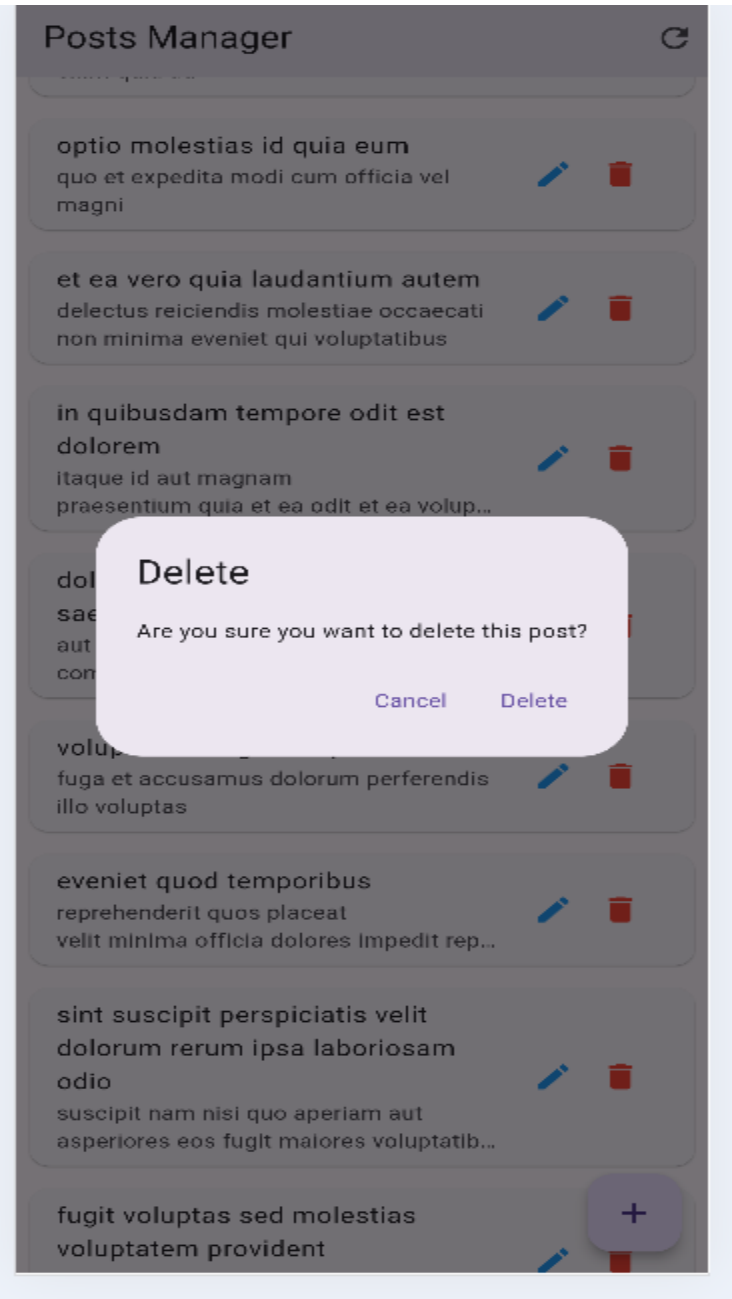
The Edit Post Screen allows users to update existing lecture data. It pre-fills the form with current lecture details, and updates the API upon submission. This illustrates data binding, state management, and API integration

2. add



This screen allows users to add a new lecture or post. The form validates inputs and sends data to the API to create a new record. This demonstrates handling user input and integrating it with API calls for data creation

3. delete



This action removes a selected lecture from the list and updates the UI immediately. It demonstrates handling user interactions, removing items from local state, and making delete requests to the API

5. API Service (api_service.dart)

```dart
class ApiService {

  // GET all posts
  Future<List<Post>> fetchPosts() async {
    final response = await http.get(Uri.parse(baseUrl));
    if (response.statusCode == 200) {
      List data = json.decode(response.body);
      return data.map((e) => Post.fromJson(e)).toList();
    } else {
      throw Exception("Failed to load posts");
    }
  }

  // CREATE a post
  Future<void> createPost(String title, String body) async {
    await http.post(
      Uri.parse(baseUrl),
      body: {"title": title, "body": body},
    );
  }

  // DELETE a post
  Future<void> deletePost(int id) async {
    await http.delete(Uri.parse("$baseUrl/$id"));
  }
}
```

This service handles all API interactions: fetching posts, creating a new post, and deleting a post. It demonstrates dependency on `http` and error handling using exceptions

Q1. Which dependencies did you use and why

> http : Used to make network request (GET, POST, DELETE) to fetch or send data to the Api.

> flutter/material.dart : provides material Design widgets for building the UI.

> No other third-party dependencies were required ~~for this~~ because all UI and state management were handled with flutter's built in tools.

Q2. Generally, how can you handle Api exceptions (network errors, invalid data)

* Try-catch block : Wrap Api calls in try {--} catch (e) {--} to catch errors.
* Check HTTP response status codes. for ex 200 means success, 400 means error
* Time out handling : Use .timeout() on request to prevent indefinite waiting
* User feedback : Display snackBar or error message if the request fails
* Valid data : ensure the response contain expected field before using it.

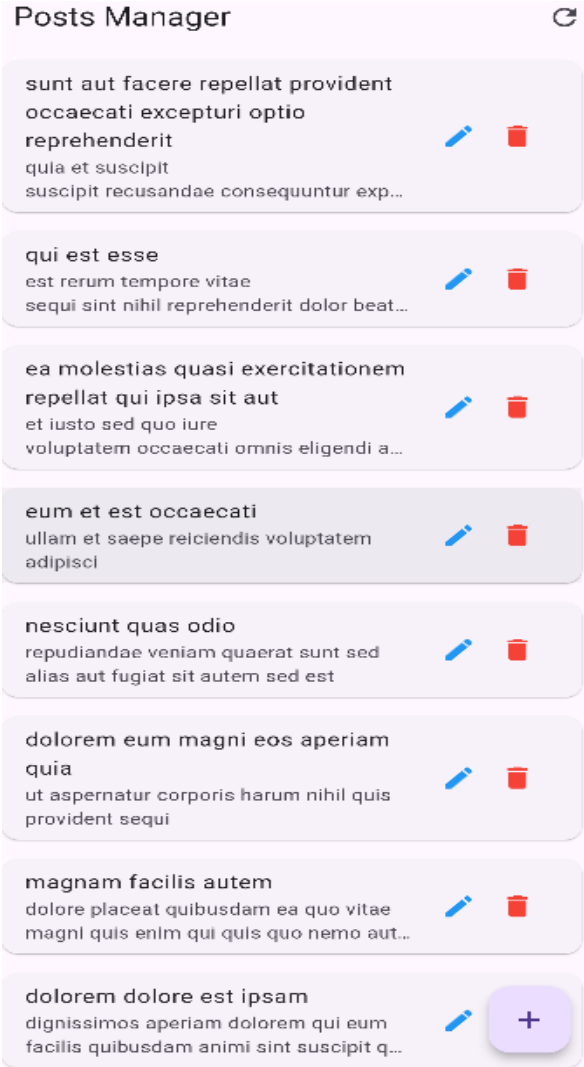
Q3. Explain the future widget you have used and the part of the UI depends on the future.

* ~~future~~ FutureBuilder is used to handle asynchronous data
  - it takes a future and builder function
  - while function is loading, it show loading indicator
  - When Future completes it show the actual data in UI
  - If it fails it show error.

CamScanner

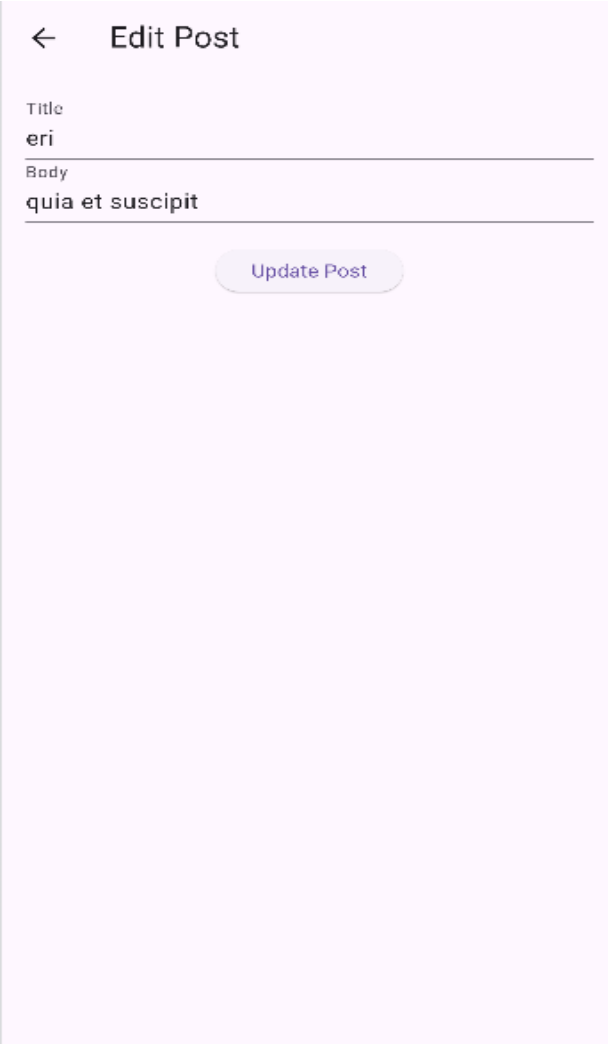
REPORTS

1. Home secreen



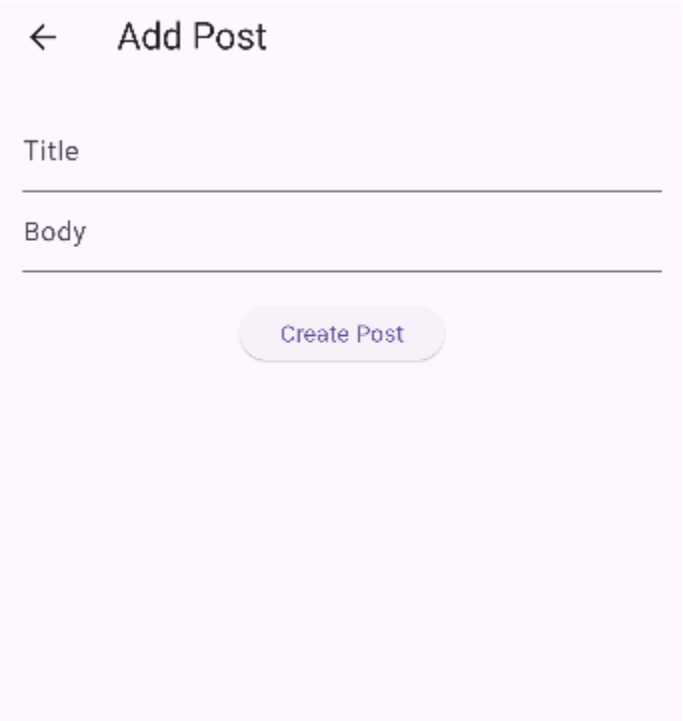
This is the Home Screen of the Lecture Manager app. It displays all lectures fetched from the API, showing the title, topic, body, and date. Users can also search lectures by title or topic. This part of the UI depends on the Future that fetches lecture data from the server

1. Edit



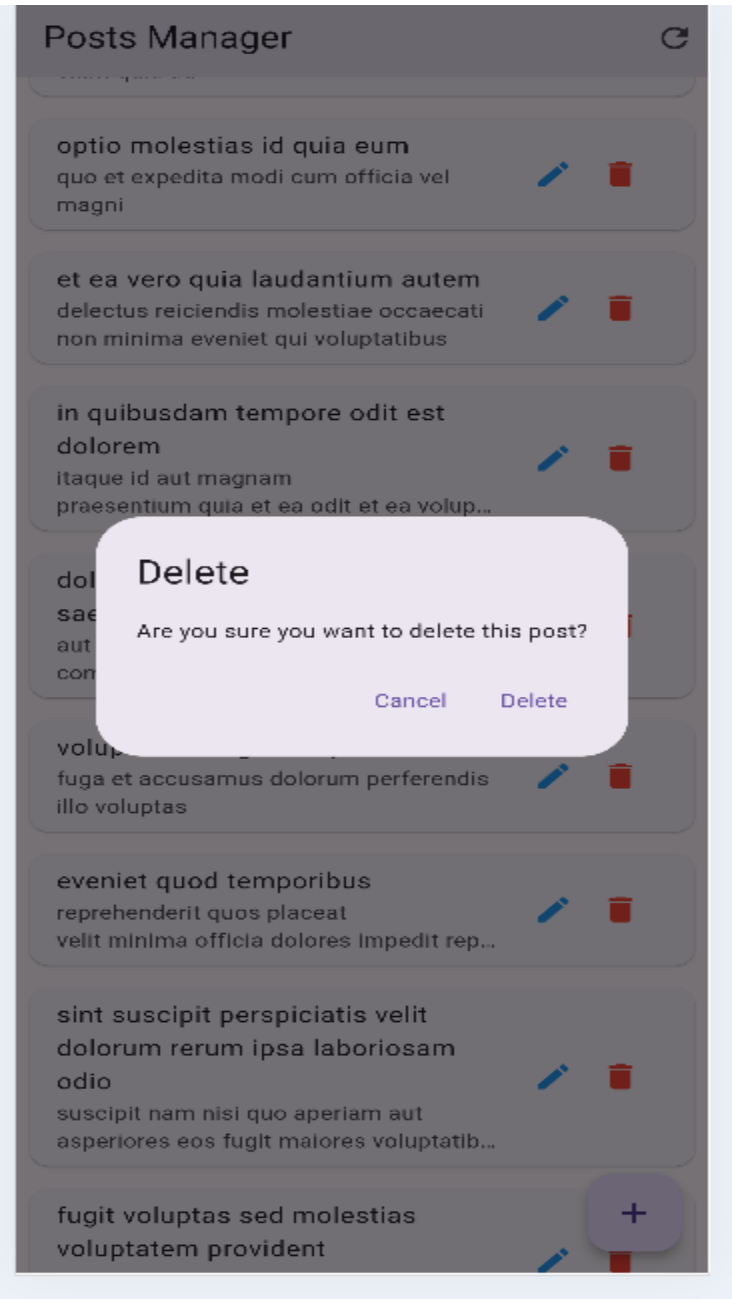
The Edit Post Screen allows users to update existing lecture data. It pre-fills the form with current lecture details, and updates the API upon submission. This illustrates data binding, state management, and API integration

2. add



This screen allows users to add a new lecture or post. The form validates inputs and sends data to the API to create a new record. This demonstrates handling user input and integrating it with API calls for data creation

3. delete



This action removes a selected lecture from the list and updates the UI immediately. It demonstrates handling user interactions, removing items from local state, and making delete requests to the API

5. API Service (api_service.dart)

```dart
class ApiService {

  // GET all posts
  Future<List<Post>> fetchPosts() async {
    final response = await http.get(Uri.parse(baseUrl));
    if (response.statusCode == 200) {
      List data = json.decode(response.body);
      return data.map((e) => Post.fromJson(e)).toList();
    } else {
      throw Exception("Failed to load posts");
    }
  }

  // CREATE a post
  Future<void> createPost(String title, String body) async {
    await http.post(
      Uri.parse(baseUrl),
      body: {"title": title, "body": body},
    );
  }

  // DELETE a post
  Future<void> deletePost(int id) async {
    await http.delete(Uri.parse("$baseUrl/$id"));
  }
}
```

This service handles all API interactions: fetching posts, creating a new post, and deleting a post. It demonstrates dependency on `http` and error handling using exceptions